

Postfix Expression Evaluation — 5 Examples

Postfix evaluation uses a **stack** and scans the expression from **left to right**.

Algorithm

For every token:

1. **Operand** → `push()` into stack.
2. **Operator**:
 - `b = pop()`
 - `a = pop()`
 - Calculate `a operator b`
 - Push the result.
3. At the end, the stack contains the final answer.

Important: For `-` and `/`, the order is always `a operator b`, not `b operator a`.

Example 1: Simple Expression

Postfix

2 3 4 * +

Equivalent infix:

2 + (3 × 4)

Evaluation

Step	Token	Action	Stack
1	2	Push 2	[2]
2	3	Push 3	[2, 3]
3	4	Push 4	[2, 3, 4]
4	*	Pop 4, 3 → <code>3 × 4 = 12</code> → Push 12	[2, 12]
5	+	Pop 12, 2 → <code>2 + 12 = 14</code> → Push 14	[14]

Answer

2 3 4 * + = 14

Example 2: Addition, Subtraction and Multiplication

Postfix

5 6 2 + * 12 4 / -

Evaluation

Step	Token	Action	Stack
1	5	Push 5	[5]
2	6	Push 6	[5, 6]
3	2	Push 2	[5, 6, 2]
4	+	6 + 2 = 8	[5, 8]
5	*	5 × 8 = 40	[40]
6	12	Push 12	[40, 12]
7	4	Push 4	[40, 12, 4]
8	/	12 ÷ 4 = 3	[40, 3]
9	-	40 - 3 = 37	[37]

Answer

5 6 2 + * 12 4 / - = 37

Equivalent infix:

(5 × (6 + 2)) - (12 / 4)

Example 3: Important Example for Operand Order

Postfix

8 2 / 3 - 4 *

This example is useful for understanding / and - .

Evaluation

Step	Token	Action	Stack
1	8	Push 8	[8]
2	2	Push 2	[8, 2]
3	/	$8 \div 2 = 4$	[4]
4	3	Push 3	[4, 3]
5	-	$4 - 3 = 1$	[1]
6	4	Push 4	[1, 4]
7	*	$1 \times 4 = 4$	[4]

Answer

8 2 / 3 - 4 * = 4

Pay attention to Step 5

When - is encountered:

Stack = [4, 3]

We pop:

b = 3
a = 4

Therefore:

a - b

$$= 4 - 3$$
$$= 1$$

Not:

$$3 - 4 = -1$$

Example 4: Modulus and Exponentiation

Postfix

$$2\ 3\ ^\wedge\ 10\ 4\ \% \ +\ 5\ ^\ast$$

Equivalent infix:

$$((2\ ^\wedge\ 3) + (10\ \% \ 4)) \times 5$$

Evaluation

Step	Token	Action	Stack
1	2	Push 2	[2]
2	3	Push 3	[2, 3]
3	^	$2^3 = 8$	[8]
4	10	Push 10	[8, 10]
5	4	Push 4	[8, 10, 4]
6	%	$10 \% 4 = 2$	[8, 2]
7	+	$8 + 2 = 10$	[10]
8	5	Push 5	[10, 5]
9	*	$10 \times 5 = 50$	[50]

Answer

$$2\ 3\ ^\wedge\ 10\ 4\ \% \ +\ 5\ ^\ast = 50$$

Example 5: Complex Expression

Let's take a more difficult expression containing nested operations.

Postfix

15 7 1 1 + - / 3 * 2 1 1 + + -

Let's evaluate carefully.

Step-by-step

Step	Token	Action	Stack
1	15	Push	[15]
2	7	Push	[15, 7]
3	1	Push	[15, 7, 1]
4	1	Push	[15, 7, 1, 1]
5	+	$1 + 1 = 2$	[15, 7, 2]
6	-	$7 - 2 = 5$	[15, 5]
7	/	$15 \div 5 = 3$	[3]
8	3	Push	[3, 3]
9	*	$3 \times 3 = 9$	[9]
10	2	Push	[9, 2]
11	1	Push	[9, 2, 1]
12	1	Push	[9, 2, 1, 1]
13	+	$1 + 1 = 2$	[9, 2, 2]
14	+	$2 + 2 = 4$	[9, 4]
15	-	$9 - 4 = 5$	[5]

Answer

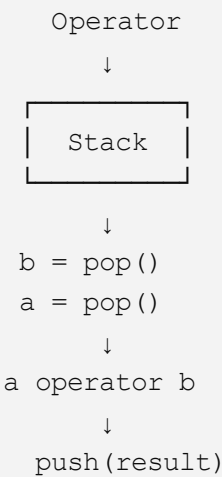
15 7 1 1 + - / 3 * 2 1 1 + + - = 5

Summary of All 5 Examples

Example	Postfix Expression	Result
1	2 3 4 * +	14
2	5 6 2 + * 12 4 / -	37
3	8 2 / 3 - 4 *	4
4	2 3 ^ 10 4 % + 5 *	50
5	15 7 1 1 + - / 3 * 2 1 1 + + -	5

The Most Important Rule

Whenever an operator is encountered:



For example:

Postfix: 10 2 3 + /

When / is encountered:

Stack = [10, 5]

Pop:

```
b = 5
a = 10
```

Calculate:

```
10 / 5 = 2
```

Push:

```
Stack = [2]
```

So:

```
10 2 3 + / = 2
```

Complexity

For `n` tokens:

```
Time Complexity  = O(n)
Space Complexity = O(n)
```

Every token is scanned once, and each operand is pushed and popped at most once.